

FundFlow — Reconciliation & Scheduler

This document explains how **operational posting**, **reconciliation**, the **scheduler**, and **system jobs** fit together. It complements the money-movement detail in [fund-flow-dynamics.md](#).

One-page summaries: [accountants](#) · [admins](#)

Related manuals: [manual-accountant.md](#) · [manual-administrator.md](#)

Table of contents

1. Big picture — three layers
 2. Realtime path on every ledger post
 3. Daily scheduler timeline
 4. Monthly collection cycle
 5. Nightly reconciliation batch
 6. Audit snapshots vs exception queue
 7. Batch posting gate
 8. Operational flows through the lifecycle
 9. System jobs UI
 10. Async queue jobs (not on cron)
 11. Summary
-

1. Big picture — three layers

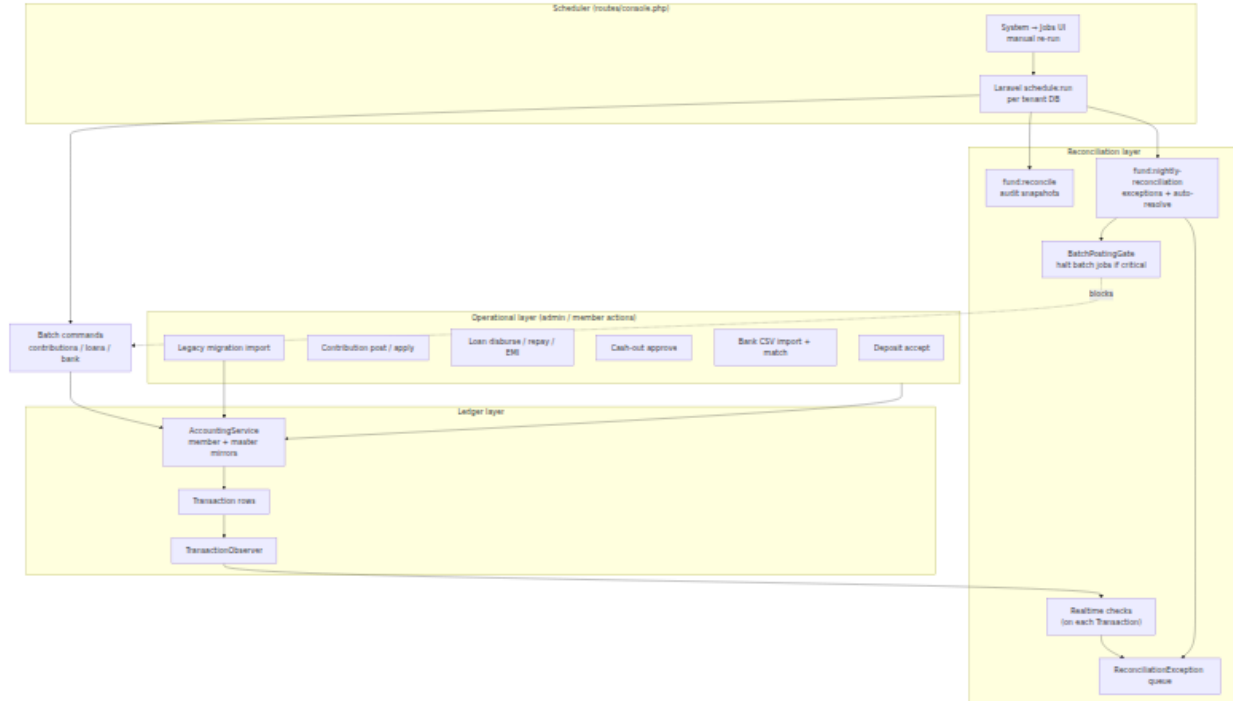


Figure 1: Diagram 1

Key idea: Operations **post intent** to the ledger immediately. Reconciliation **verifies** that intent (realtime

+ nightly). The scheduler **drives recurring collection and checks** on a calendar. They are separate steps — bank clearance, for example, does not re-post cash when you match a line.

Layer	What it does	When it runs
Operational	Admin/member actions (accept deposit, apply contribution, etc.)	On demand
Ledger	Mirrored journal entries via <code>AccountingService</code>	Same transaction as the action
Realtime recon	Lightweight guards on each <code>Transaction</code>	Immediately after post
Snapshots	Historical audit report stored in <code>reconciliation_snapshots</code>	Daily 06:20, monthly 2nd
Nightly batch	Full domain sweeps, auto-resolve, halt gate	Daily 06:30
Scheduler	Recurring collection, fees, bank match, statements	Calendar in <code>routes/console.php</code>

2. Realtime path on every ledger post

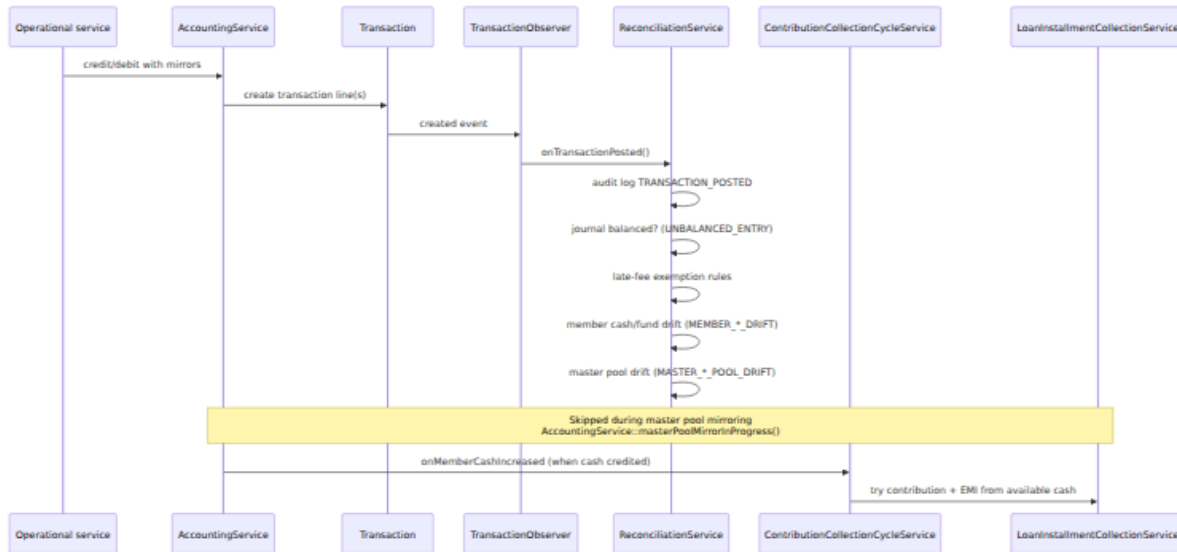


Figure 2: Diagram 2

Realtime checks (`ReconciliationService::onTransactionPosted()`):

Check	Exception code	Severity
Journal DR = CR for reference	<code>UNBALANCED_ENTRY</code>	critical
Late-fee exemption consistency	varies	varies
Member component formula vs stored balance	<code>MEMBER_CASH_DRIFT</code> , <code>MEMBER_FUND_DRIFT</code>	high

Check	Exception code	Severity
Master cash/fund vs Σ members	MASTER_CASH_POOL_DRIFT, MASTER_FUND_POOL_DRIFT	high

Skipped when:

- `ReconciliationService::realtimeChecksSuspended()` is active
- `AccountingService::masterPoolMirrorInProgress()` — paired master/member legs post in one logical operation

Heavy domain sweeps (contributions past window, bank uncleared pipeline, EMI state) run in the **nightly batch**, not on every transaction.

3. Daily scheduler timeline

All scheduled tenant commands use `TenantAwareScheduledCommand` — `schedule:run` executes each command **once per tenant database**.

Source: `routes/console.php` · Registry: `App\Support\ScheduledJobRegistry`

Time	Command	Role	Halt-sensitive
06:00	<code>fund:assert-master-invariant</code>	Quartz gate: master cash/fund = Σ members	No
06:20	<code>fund:reconcile --daily</code>	Audit snapshot for yesterday	No
06:30	<code>fund:nightly-reconciliation</code>	Control layer: exceptions, auto-resolve, halt gate	No
07:00	<code>loans:check-defaults</code>	Delinquency / guarantor / auto-transfer	No
07:15	<code>contributions:apply-late-fees</code>	Late fees contribution + EMI late fees	Yes
07:30	<code>delinquency:send-digest</code>	Admin digest notification	No
08:00	<code>bank:auto-match</code>	Match imported bank lines uncleared postings	Yes
<i>every minute</i>	<code>announcements:dispatch-scheduled</code>	Scheduled member announcements	No

Manual run: Audit & System → Jobs — same commands, subject to `BatchPostingGate` for halt-sensitive entries.

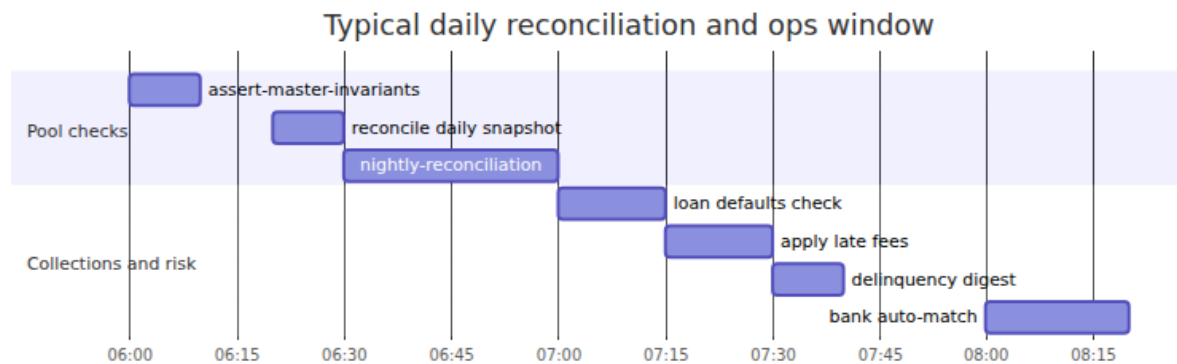
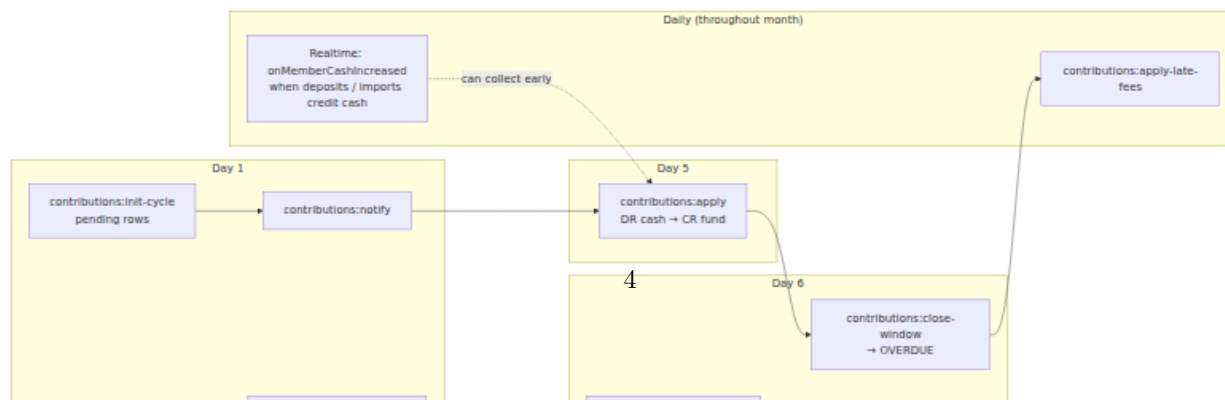


Figure 3: Diagram 3

4. Monthly collection cycle

Day	Time	Command	Role	Halt-sensitive
1st	08:00	contributions:init-cycle	Create pending contribution rows	Yes
1st	08:00	loans:send-due-notifications	File notifications	No
1st	09:00	contributions:notify	Contribution due notifications	No
2nd	06:30	fund:reconcile --monthly	Previous month audit snapshot	No
3rd	08:00	statements:generate --notify	Monthly statements + notify	No
5th	09:00	contributions:apply	Batch debit cash → credit fund	Yes
6th	00:30	contributions:close-window	Unpaid → overdue	Yes
6th	00:45	loans:close-emi-window	Unpaid installments → overdue	Yes
6th	06:00	loans:apply-repayments	Batch EMI collection from cash	Yes



5. Nightly reconciliation batch

`fund:nightly-reconciliation` → `ReconciliationService::runNightlyBatch()`:

Sweep	Typical exceptions raised
Master pre-check	MASTER_IMBALANCE_UNRESOLVED (critical → halt)
Contributions	PENDING_PAST_WINDOW_CLOSE, COLLECTED_WITHOUT_POST, tier mismatches
Loans / EMI	EMI_COLLECTED_LEDGER_MISSING, ACTIVE_BEFORE_FULL_DISBURSE, schedule state
Fund tiers	Tier / fund allocation consistency
Bank clearing	RECON_UNMATCHED_BANK_LINE, UNMATCHED_CASH_ENTRY, STALE_PENDING
Late fees	FEE_WRONG_TIER, FEE_INCOME_DRIFT
Member invariants	MEMBER_CASH_DRIFT, MEMBER_FUND_DRIFT

After domain sweeps, **auto-resolve** attempts fix simple cases. A **post-batch** master balance check may halt again if drift remains critical.

6. Audit snapshots vs exception queue

These are **different products** — do not confuse them.

Mechanism	Command	Output	Use
Exception queue	<code>fund:nightly-reconciliation</code>	live <code>reconciliation_exceptions</code> rows	Active mediation in admin UI
Audit snapshot	<code>fund:reconcile --daily</code> <code>/ --monthly</code>	<code>reconciliation_snapshot</code> + PDF metrics	Historical audit trail for accountants
Point-in-time audit	<code>fund:reconcile</code> <code>--realtime</code>	Report only (<code>--no-store</code> skips save)	Ad-hoc investigation

Snapshots include ledger mismatch counts, unposted bank rows, open exception counts, and coverage matrices — useful for month-end packs, not for replacing the live queue.

7. Batch posting gate

Triggers halt:

- MASTER_IMBALANCE_UNRESOLVED critical exception (open or raised during nightly batch)
- Manual halt via system settings (`batch_posting_halted`)

Enforcement:

- `ScheduledJobRegistry` — `halt_sensitive: true` on affected jobs
- `SystemJobRunnerService` — blocks manual runs from Jobs UI
- `EnsuresBatchPostingAllowed` trait — blocks Artisan command execution

Recovery:

1. Fix underlying master/member pool drift.
2. Resolve or write off critical exceptions.

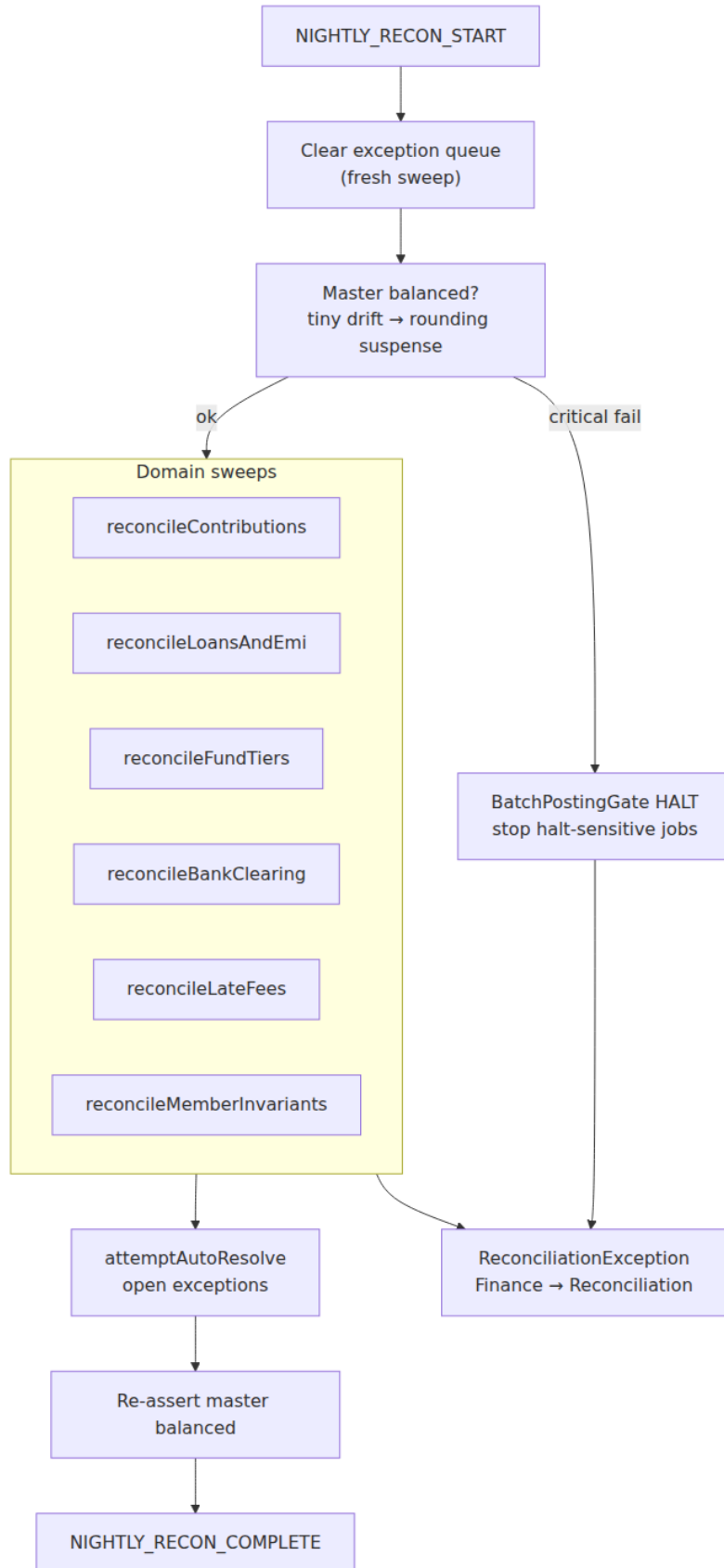


Figure 5: Diagram 5

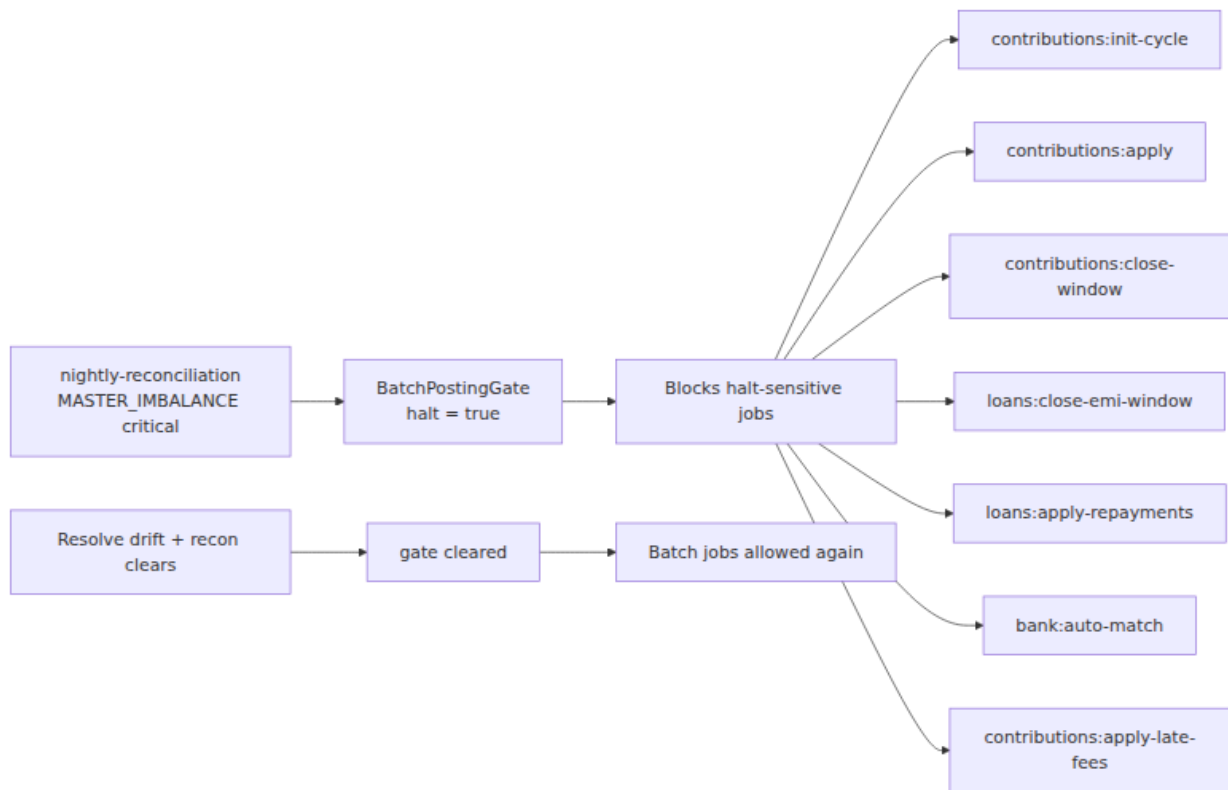


Figure 6: Diagram 6

3. Re-run `fund:nightly-reconciliation` (clears gate on success), or clear halt in settings after verification.

Not blocked: deposit accept, manual corrections, snapshot commands, `fund:assert-master-invariants`, notifications, delinquency checks.

8. Operational flows through the lifecycle

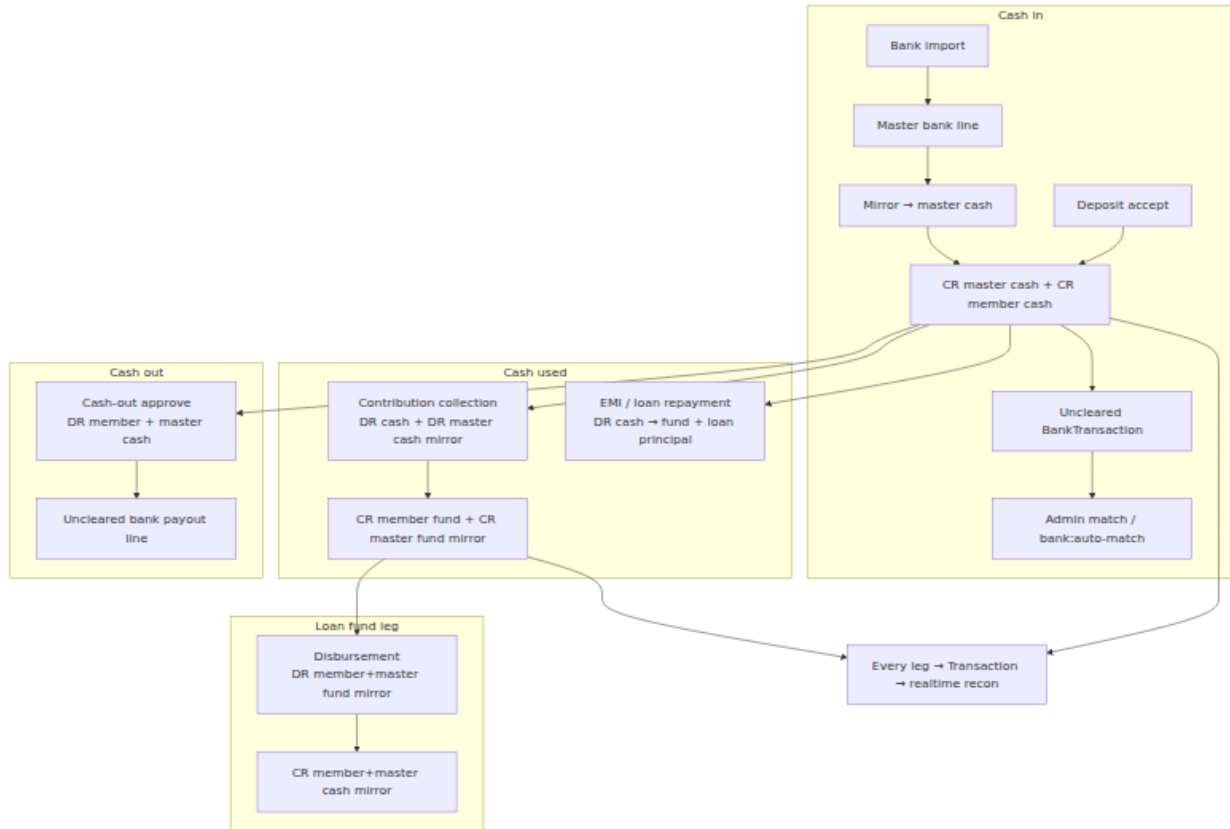


Figure 7: Diagram 7

Every arrow that creates a **Transaction** feeds the realtime reconciliation path (section 2). Bank **match/clear** updates linkage — it does **not** post additional cash legs when done correctly.

9. System jobs UI

Location: Audit & System → Jobs

Feature	Behaviour
Job list	Mirrors <code>ScheduledJobRegistry::all()</code>
Manual run	<code>SystemJobRunnerService::run()</code> — records <code>SystemJobRun</code>
Halt badge	Shown when <code>BatchPostingGate::isHalted()</code>
History	Per-run exit code, duration, output snippet

Scheduled cron and manual runs use the **same Artisan commands** — only the trigger differs (`SystemJobRun::TRIGGER_SCHEDULED` vs `TRIGGER_MANUAL`).

10. Async queue jobs (not on cron)

Job	Trigger	Purpose
<code>RunLegacyMigrationPaymentsJob</code>	Legacy Migration UI	Classify + import historical payments
<code>ClassifyLegacyPaymentsJob</code>	Legacy Migration UI	Payment classification only
Tenant provisioning jobs	Central app	Cache dirs, tenant setup

Legacy migration runs **outside** the nightly scheduler. After import, operators may run repair commands (`legacy:repair-excess-loan-repayments`, `accounting:rebuild-balances`, etc.) before relying on reconciliation results.

11. Summary

FundFlow separates **posting** (what happened economically), **clearance** (does the bank agree?), and **reconciliation** (do the books still obey pool rules?).

1. Day-to-day actions post mirrored ledger entries immediately.
2. Each **Transaction** triggers lightweight realtime checks.
3. Every morning the scheduler runs pool assertions, stores an audit snapshot, then runs a full exception sweep that can halt batch collection jobs if the master pool is broken.
4. Monthly commands open and close contribution/EMI windows; between those dates, cash credits still drive automatic collection via `onMemberCashIncreased`.
5. Admins can re-run any scheduled job from **System** → **Jobs**, subject to the batch halt gate.

References

Resource	Path
Scheduler definition	<code>routes/console.php</code>
Job registry	<code>app/Support/ScheduledJobRegistry.php</code>
Nightly batch	<code>app/Services/ReconciliationService.php</code>
Halt gate	<code>app/Support/BatchPostingGate.php</code>
Manual job runner	<code>app/Services/SystemJobRunnerService.php</code>
Mirror rules	<code>.cursor/rules/accounting-master-member-sync.mdc</code>
Accountant manual	<code>docs/manual-accountant.md</code>
Administrator manual	<code>docs/manual-administrator.md</code>